

LAPORAN MACHINE LEARNING



Disusun Oleh:

Nama : Aisyah Azzahrah
NPM : G1A024033

Kelas:

Informatika (A)

Dosen Pengampu:

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

DESKRIPSI TUGAS

Kamu adalah seorang Data Scientist di startup EduTech. Perusahaan ingin memprediksi "Apakah mahasiswa akan LULUS TEPAT WAKTU atau TIDAK".

Dataset: mahasiswa_lulus.csv - 500 data dengan fitur: IPK, Kehadiran, Jam_Belajar, Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa

Tugas kamu ada 3 Bagian:

BAGIAN A: ANALISIS KONSEP

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawaban :

Decision Tree lebih cocok digunakan ketika hubungan antar fitur (IPK, Kehadiran, Jam_Belajar, Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa) saling berinteraksi dan tidak mengikuti pola statistik sederhana, misalnya mahasiswa dengan IPK tinggi tetapi kehadiran rendah dapat memiliki hasil berbeda dari mahasiswa yang keduanya tinggi. Decision Tree menangkap interaksi tersebut melalui aturan bercabang.

Alasan kedua, Decision Tree lebih mudah diinterpretasikan karena hasilnya berupa pohon keputusan yang menunjukkan fitur dan aturan yang digunakan, berbeda dengan Naive Bayes yang berbasis probabilitas dan mengasumsikan setiap fitur independen. Jika tujuan utamanya adalah model yang mudah dijelaskan kepada kampus dan mampu menangkap interaksi antarfitur, Decision Tree lebih sesuai.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawaban :

Naive Bayes mengasumsikan setiap fitur independen secara kondisional terhadap kelas yang diprediksi. Pada dataset ini, asumsi tersebut kemungkinan dilanggar karena beberapa karakteristik mahasiswa saling berkaitan.

Pasangan pertama adalah IPK dan Jam_Belajar, karena mahasiswa dengan jam belajar tinggi cenderung memperoleh IPK lebih tinggi. Pasangan kedua adalah Penghasilan_Ortu dan Status_Beasiswa, karena kondisi ekonomi orang tua berkaitan dengan peluang memperoleh beasiswa. Hubungan ini membuat asumsi independensi Naive Bayes tidak sepenuhnya terpenuhi.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawaban :

Overfitting terjadi ketika pohon keputusan terlalu kompleks dan menyesuaikan diri dengan data training. Dengan hanya 500 data, `max_depth=None` memungkinkan pohon tumbuh sangat dalam dan mempelajari noise, sehingga akurasi tinggi di training tetapi menurun pada data baru.

Cara mengatasinya antara lain membatasi kedalaman pohon (misalnya `max_depth=3` seperti diminta dalam tugas), menggunakan `min_samples_split` atau `min_samples_leaf`, menerapkan pruning, serta selalu mengevaluasi model pada data testing agar model benar-benar mampu melakukan generalisasi.

BAGIAN B : IMPLEMENTASI & EVALUASI

Gunakan Python + sklearn + pandas + matplotlib

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

Jawaban:

Source Code :

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load data
data = pd.read_csv('dataset.csv')

# Check for missing values
print(data.isnull().sum())

# Handle missing values
data = data.fillna(-1)

# Encode categorical variables
encoder = OneHotEncoder(sparse=False)
categorical_columns = data.columns[data.dtypes == 'object']
encoded_data = pd.DataFrame(encoder.fit_transform(data[categorical_columns]).toarray(),
                             columns=encoder.get_feature_names_out(categorical_columns))

# Scale numerical variables
scaler = StandardScaler()
numerical_columns = data.columns[data.dtypes != 'object']
scaled_data = pd.DataFrame(scaler.fit_transform(data[numerical_columns]),
                             columns=numerical_columns)

# Combine encoded and scaled data
final_data = pd.concat([scaled_data, encoded_data], axis=1)

# Train-test split
X = final_data.drop('Status_Berisiko', axis=1)
y = final_data['Status_Berisiko']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# Decision Tree Model
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=3, min_samples_split=10, min_samples_leaf=5)
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

print(f'Accuracy: {accuracy}')
print(conf_matrix)
print(class_report)

```

Output :

```

1. DATA PENDING
Jumlah baris x kolom : (500, 8)

Info type data & missing value:
IDN      500
Kebadian  500
Jus_Belajar  500
Organisasi  500
Penghasilan_Drta  0
Beris_Kelompok  0
Status_Berisiko  0
Lulus_Tegat_Makto  0
dtype: object

Contoh 5 baris pertama:
IDN Kebadian Jus_Belajar Organisasi Penghasilan_Drta Beris_Kelompok Status_Berisiko
0 3.32 83.1 5.3 Aktif Penangan Perencanaan
1 3.34 100.0 5.7 Tidak Aktif Rendah Perencanaan
2 3.36 89.2 3.3 Tidak Aktif Penangan Perencanaan
3 3.70 88.8 2.5 Aktif Rendah Laki-laki
4 3.99 74.2 4.8 Aktif Penangan Laki-laki

```

```

Status_Berisiko Lulus_Tegat_Makto
0      Ya      Tegat Makto
1      Tidak Tegat Makto
2      Tidak Tegat Makto
3      Ya      Tegat Makto
4      Tidak Tegat Makto

2. PREPROCCESING

Missing 'IDN' diisi dengan median = 3.38
Missing 'Kebadian' diisi dengan median = 82.3
Missing 'Jus_Belajar' diisi dengan median = 3.7
Missing 'Organisasi' diisi dengan median = Tidak Aktif

Total missing value setelah dihandle: 0
Encoding 'Organisasi': ['Aktif': np.int64(0), 'Tidak Aktif': np.int64(1)]
Encoding 'Penghasilan_Drta': ['Menengah': np.int64(0), 'Rendah': np.int64(1), 'Tinggi': np.int64(2)]
Encoding 'Beris_Kelompok': ['Laki-laki': np.int64(0), 'Perempuan': np.int64(1)]
Encoding 'Status_Berisiko': ['Tidak': np.int64(0), 'Ya': np.int64(1)]
Encoding Target 'Lulus_Tegat_Makto': ['Tegat Makto': np.int64(0), 'Tidak Tegat Makto': np.int64(1)]

Jumlah data latih : 400
Jumlah data uji : 100

```

Penjelasan :

Pemrosesan data dimulai dari penghubung Colab ke Drive, pemuatan dataset 500 baris, penanganan *missing value*, hingga *encoding* dan *train-test split* 80:20. Selanjutnya, tiga model (*Decision Tree* `max_depth=3`, `max_depth=None`, dan

Gaussian Naive Bayes) dilatih serta dievaluasi menggunakan metrik akurasi, *precision*, *recall*, *F1-score*, dan *confusion matrix*. Terakhir, dibuat visualisasi pohon keputusan serta diagram batang metrik, disertai analisis error pada 5 data uji yang salah diprediksi.

2. Modeling:

Bangun model DecisionTreeClassifier dengan max_depth=3 dan max_depth=None

Bangun model GaussianNB

Jawabana :

Source Code dan Output :

```

# 2. MODELING
# 2.1. Decision Tree
# 2.1.1. max_depth=3
from sklearn.tree import DecisionTreeClassifier
max_depth=3, random_state=42
model = DecisionTreeClassifier(max_depth=max_depth, random_state=random_state)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1_score = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy}, Precision: {precision}, Recall: {recall}, F1-Score: {f1_score}")

# 2.1.2. max_depth=None
from sklearn.tree import DecisionTreeClassifier
max_depth=None, random_state=42
model = DecisionTreeClassifier(max_depth=max_depth, random_state=random_state)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1_score = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy}, Precision: {precision}, Recall: {recall}, F1-Score: {f1_score}")

# 2.2. Gaussian Naive Bayes
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1_score = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy}, Precision: {precision}, Recall: {recall}, F1-Score: {f1_score}")

```

```

2. MODELING
=====
Model 'Decision Tree (max_depth=3)' selesai dilatih.
Model 'Decision Tree (max_depth=None)' selesai dilatih.
Model 'Gaussian Naive Bayes' selesai dilatih.

```

Penjelasan :

Tahap *modeling* menginisialisasi dan melatih tiga algoritma klasifikasi—*Decision Tree* (max_depth=3), *Decision Tree* tanpa batas (max_depth=None), dan *Gaussian Naive Bayes*—menggunakan data latih, lalu menghasilkan prediksi pada data uji secara otomatis melalui perulangan (*looping*).

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model.

Buat Confusion Matrix

Jawaban :

Source Code Output :

```

# 3. EVALUATION
# 3.1. Accuracy, Precision, Recall, F1-Score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1_score = f1_score(y_test, y_pred)
print(f"Accuracy: {accuracy}, Precision: {precision}, Recall: {recall}, F1-Score: {f1_score}")

# 3.2. Confusion Matrix
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

```

```

# 3.2. Confusion Matrix
# 3.2.1. Decision Tree (max_depth=3)
cm = confusion_matrix(y_test, y_pred)
print(cm)

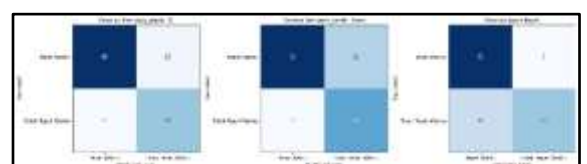
# 3.2.2. Decision Tree (max_depth=None)
cm = confusion_matrix(y_test, y_pred)
print(cm)

# 3.2.3. Gaussian Naive Bayes
cm = confusion_matrix(y_test, y_pred)
print(cm)

```

Classification Report					
	precision	recall	f1-score	support	
0	0.77	0.83	0.80	30	
1	0.87	0.86	0.86	30	
accuracy	0.80	0.79	0.79	60	
weighted avg	0.80	0.79	0.79	60	

	precision	recall	f1-score
Decision Tree (max_depth=3)	0.77	0.83	0.80
Decision Tree (max_depth=None)	0.86	0.86	0.86
Gaussian Naive Bayes	0.77	0.80	0.78



Penjelasan :

Tahap evaluasi menguji kinerja ketiga model menggunakan metrik *Accuracy*, *Precision*, *Recall*, dan *F1-Score*, lengkap dengan laporan klasifikasi dan penyimpanan *Confusion Matrix*. Berdasarkan tabel ringkasan metrik, *Gaussian Naive Bayes* memperoleh akurasi tertinggi sebesar 79% (F1-Score 0.6957), diikuti oleh *Decision Tree* (*max_depth=3*) sebesar 71%, dan *Decision Tree* (*max_depth=None*) terendah di angka 66% akibat kecenderungan *overfitting*.

4. Visualisasi:

Plot Pohon Keputusan dengan *max_depth=3*

Plot perbandingan metrik 3 model dalam bentuk bar chart

Jawaban :

Source Code dan Output :

```

# 1. Importasi library
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix

# 2. Load data
data = pd.read_csv('data.csv')

# 3. Preprocessing
X = data[['Pemeriksaan', 'Gejala', 'Durasi', 'Tingkat', 'Status', 'Organisme']]
y = data['Klasifikasi']

# 4. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Model Training
# Gaussian Naive Bayes
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Decision Tree
dt = DecisionTreeClassifier(max_depth=3)
dt.fit(X_train, y_train)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, max_depth=3)
rf.fit(X_train, y_train)

# 6. Model Evaluation
# Gaussian Naive Bayes
y_pred_gnb = gnb.predict(X_test)
acc_gnb = accuracy_score(y_test, y_pred_gnb)
prec_gnb = precision_score(y_test, y_pred_gnb)
rec_gnb = recall_score(y_test, y_pred_gnb)
f1_gnb = f1_score(y_test, y_pred_gnb)

# Decision Tree
y_pred_dt = dt.predict(X_test)
acc_dt = accuracy_score(y_test, y_pred_dt)
prec_dt = precision_score(y_test, y_pred_dt)
rec_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)

# Random Forest
y_pred_rf = rf.predict(X_test)
acc_rf = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf)
rec_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)

# 7. Confusion Matrix
cm_gnb = confusion_matrix(y_test, y_pred_gnb)
cm_dt = confusion_matrix(y_test, y_pred_dt)
cm_rf = confusion_matrix(y_test, y_pred_rf)

# 8. Visualization
# Plot Pohon Keputusan
plt.figure(figsize=(10,10))
dt.feature_importances_
dt.plot_tree(class_labels=y_test.unique(), feature_labels=X.columns)
plt.savefig('decision_tree_plot.png')

# Bar chart perbandingan metrik
plt.figure(figsize=(10,10))
models = ['Gaussian Naive Bayes', 'Decision Tree (max_depth=3)', 'Random Forest (max_depth=3)']
metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
values = [[acc_gnb, acc_dt, acc_rf], [prec_gnb, prec_dt, prec_rf], [rec_gnb, rec_dt, rec_rf], [f1_gnb, f1_dt, f1_rf]]
plt.bar(metrics, values)
plt.savefig('perbandingan_metrik.png')

# Feature Importance
plt.figure(figsize=(10,10))
feature_importances = dt.feature_importances_
plt.bar(X.columns, feature_importances)
plt.savefig('feature_importance.png')

```

```

# 9. Feature Importance
plt.figure(figsize=(10,10))
feature_importances = dt.feature_importances_
plt.bar(X.columns, feature_importances)
plt.savefig('feature_importance.png')

```

=====

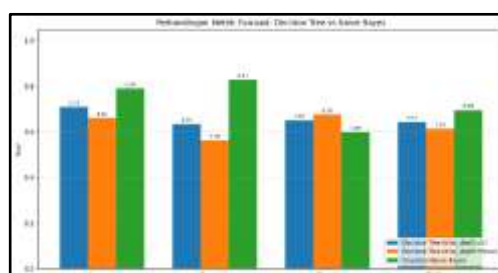
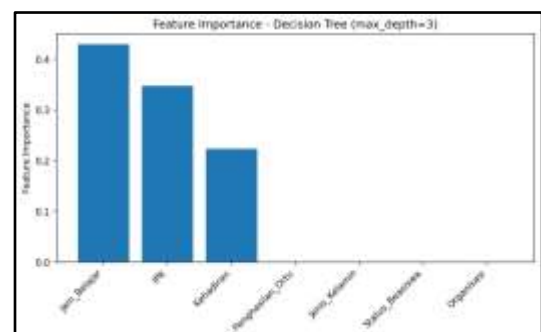
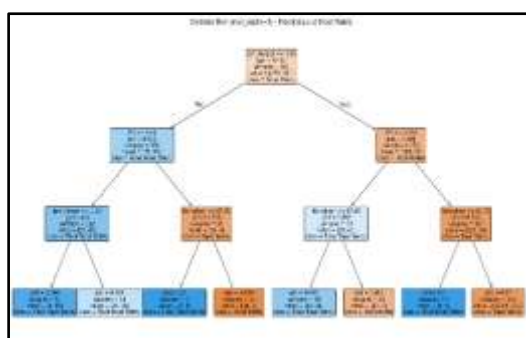
5. VISUALISASI

=====

Plot pohon keputusan disimpan -> decision_tree_plot.png

Bar chart perbandingan metrik disimpan -> perbandingan_metrik.png

Feature importance disimpan -> feature_importance.png



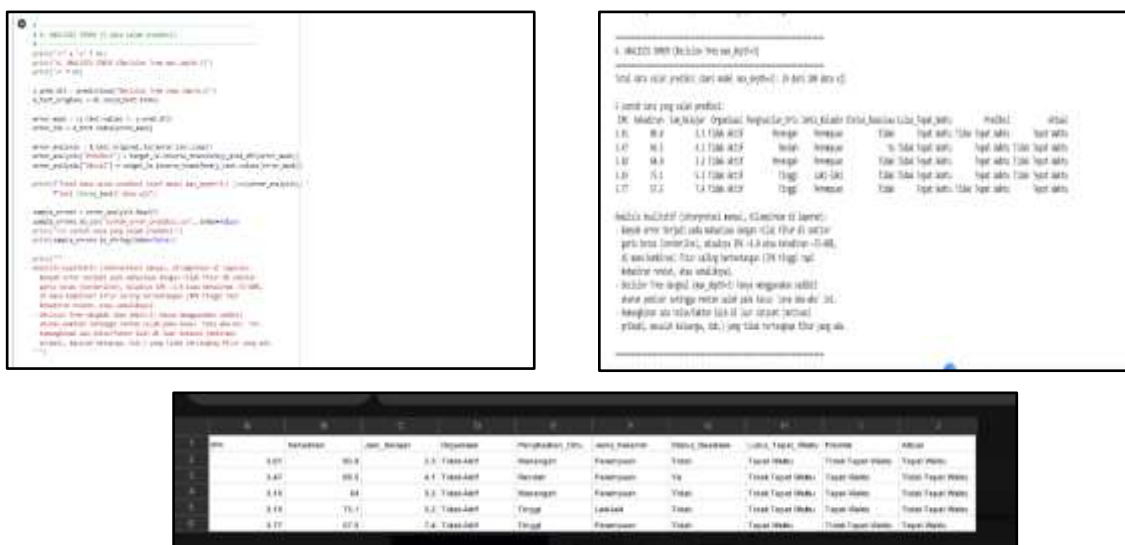
Penjelasan :

Tahap visualisasi menghasilkan diagram pohon keputusan (*plot_tree*) untuk melihat aturan model berkedalaman 3 secara transparan, diagram batang (*bar chart*) untuk membandingkan metrik antar model secara visual, serta grafik *feature importance* guna mengidentifikasi fitur yang paling berpengaruh dalam menentukan prediksi kelulusan mahasiswa.

5. Analisis Error: Ambil 5 data yang salah prediksi. Jelaskan kenapa menurutmu model salah

Jawaban :

Source Code dan Output :



Penjelasan :

Tahap analisis error mengidentifikasi 5 sampel data uji dengan prediksi kelulusan yang keliru dari model *Decision Tree* (*max_depth=3*). Kesalahan prediksi ini umumnya terjadi pada data *borderline* atau zona abu-abu, seperti kombinasi nilai IPK tinggi namun tingkat kehadiran atau jam belajar yang rendah, sehingga batasan aturan model gagal menangkap kompleksitas kondisi aktual tersebut.

BAGIAN C : REKOMENDASI & ETIKA

Buat Laporan + Slide Presentasi 5 slide

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus?

Jelaskan dengan data dari Bagian B

Jawaban :

Model yang Dipilih: Gaussian Naive Bayes

Penjelasan :

- 1) Akurasi Tertinggi: Berdasarkan tabel evaluasi, *Gaussian Naive Bayes* mencatatkan Akurasi 79% dan Precision 83%, yang merupakan performa tertinggi dibandingkan *Decision Tree (max_depth=3)* sebesar 71% dan *Decision Tree (max_depth=None)* sebesar 66%.
- 2) Menghindari Overfitting: Model *Decision Tree* dengan `max_depth=None` terbukti mengalami penurunan performa (overfitting) pada data uji. Oleh karena itu, *Gaussian Naive Bayes* lebih konsisten dan andal untuk diterapkan dalam memprediksi kelulusan mahasiswa secara objektif.

https://drive.google.com/file/d/14U_PSiNf2pnLtP5cCSUug_Z3FUKITciD/view?usp=sharing

2. Bias & Etika: Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawaban :

Dampak Jika Dipakai untuk Beasiswa: Fitur seperti "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan memicu diskriminasi sistemik. Jika model digunakan untuk seleksi beasiswa, sistem berisiko mempelajari bias historis (misalnya, memperlakukan tidak adil kelompok gender tertentu atau mendiskriminasi mahasiswa dari tingkat penghasilan tertentu), sehingga keputusan beasiswa menjadi tidak objektif dan tidak adil.

Cara Mitigasinya:

- 1) Melakukan Fairness Audit secara berkala untuk mengukur tingkat keadilan model terhadap kelompok demografis yang berbeda.
 - 2) Mempertimbangkan penghapusan fitur sensitif (*feature dropping*) jika fitur tersebut terbukti memicu bias tanpa relevansi langsung terhadap kapasitas akademik murni.
 - 3) Menjadikan hasil prediksi model hanya sebagai alat bantu keputusan (*decision support tool*), bukan sebagai penentu mutlak tunggal dalam kelayakan beasiswa.
3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda.

Jawaban :

Source Code dan Output :

